

数据结构专题

A_zjzj 郑钧

北京大学

2026 年 2 月 26 日

P11160 【MX-X6-T6】机械生命体

题目描述

维护一个可重集 S ，初始为空，共有 q 次操作。操作方式如下：

- 1 x ，你需要在 S 中加入一个数 x ；
- 2 x ，你需要在 S 中删除一个数 x 。保证此时 S 中存在至少一个 x 。如果存在多个 x ，则仅删除一个；
- 3 x k v ，你需要对 S 中所有满足 $\text{lowbit}(x \oplus y) \geq 2^k$ 的 y 增加 v 并对 2^{32} 取模；
- 4 x ，你需要求出 $\max_{y \in S} \text{lowbit}(x \oplus y)$ 。保证此时 S 不为空。

$1 \leq q \leq 5 \times 10^5$, $0 \leq x, y, v \leq 2^{32} - 1$, $0 \leq k \leq 32$ 。

解法思路

- 使用 01-Trie 维护，本质上和动态开点线段树没有任何区别，但是这题需要从低位到高位维护。
- 操作 1, 2, 4 是简单的，下面考虑操作 3 怎么做。
- 难点在于需要 $+v$ ，如果是 $\oplus v$ 的话，可以直接打 tag，然后做一遍线段树合并即可。
- $+v$ 其实也是一样的，打一个 tag，表示这个子树的所有元素都需要加上一个值，最高位直接溢出。
- 那么下推标记的操作其实也是容易维护的，如果 tag_u 是奇数，那么就交换左右儿子，新的左儿子需要加上 $(tag_u + 1)/2$ ，新的右儿子加上 $(tag_u - 1)/2$ ；否则两边都加上 $tag_u/2$ 即可。
- 时空复杂度 $O(q \log V)$ 。

CF464E The Classic Problem

题目描述

有一个 n 个顶点 m 条边的有向图，每条边的边权为 2^{w_i} ，求出 s 到 t 的最短路。

$1 \leq n \leq 10^5$, $0 \leq m \leq 10^5$, $0 \leq w_i \leq 10^5$ 。

解法思路

- 显然会使用 dijkstra 解决单源最短路问题，每个点的距离是一个很大的数，考虑用主席树维护其二进制的表示。
- 然后 dijkstra 需要支持比较两个数的大小关系，可以使用线段树上二分加哈希解决。
- 加 2^{w_i} 的操作，只需要二分到 w_i 以及更高位中第一个 0，之前的 1 覆盖为 0 即可。
- 需要注意可持久化，时空复杂度 $O(n \log n)$ 。

洛谷 P4747 [CERC2017] Intrinsic Interval

题目描述

给定 $1 \sim n$ 的排列 $p_1, p_2, \dots, p_n$, 定义一个区间 $[l, r]$ 是好的当且仅当 $p_l, \dots, p_r$ 在值域上也是一段区间。

m 次询问, 每次询问给定 x, y , 求出包含区间 $[x, y]$ 最小的好的区间 (可以证明这样的区间是存在且唯一的)。

$1 \leq n, m \leq 10^5$ 。

解法思路

- 先考虑一个简单一点的问题，如何求出好的区间的个数和。
- 这个问题可以使用扫描线加线段树解决的，再考虑这个做法能不能解决原来的问题。
- 发现是可以的，只需要从后往前扫描 r ，维护每个 l 的最大的长度，然后查询 l 的一个前缀的最小值即可。

LOJ 4261. 「KTSC 2024 R2」跳跃游戏

题目描述

序列 $A_{0 \sim N-1}$ 由如下方式生成：

- 初始 A 全为 0；
- Q 次操作，每次把 A 的一个区间 $[L_i, R_i]$ 全部加一。

选出 A 的一些元素，要求任何两个选中的元素之间的下标差不得小于 K ，求出元素和的最大值。

$1 \leq N \leq 10^{12}$, $1 \leq Q \leq 2.5 \times 10^5$, $1 \leq K \leq N$, $0 \leq L_i \leq R_i \leq N-1$ 。

解法思路

- 先考虑 $O(N)$ 的做法, 设 f_i 表示 dp 到元素 i , 此时的最大值是多少, 转移方程如下:

$$f_i = \max\{f_{i-1}, f_{i-K} + A_i\}$$

- 然后我们考虑把 f 滚动一下, 即我们不需要存储非常多的 f , 我们只需要存储 $f_{i-K \sim i-1}$ 即可, 因此我们可以把数组 f' 的大小设置为 K , f_i 的值就存储在 $f'_{i \bmod K}$ 中。
- 那么此时只需要进行转移:

$$f'_{i \bmod K} \leftarrow f'_{(i-1) \bmod K} + A_i$$

解法思路

- 如果 $K \leq 2.5 \times 10^5$ 的时候怎么做？
- 考虑用线段树维护 f' ，那么可以一次性做很多次转移：先把 $f'_{l \sim r}$ 整体加 A_i ，再整体和 f'_{l-1} 取 $\max$ 。（此处用到了 f 的单调性）
- 那么我们考虑将一整段 A_i 相同的 i 一块做。
- 接着我们发现，这一整段的转移，只需要转移至多 K 个位置，剩下的转移只需要加 A_i 即可。（可以从 dp 数组考虑，也可以从贪心角度考虑）
- 最后，扩展到 $K \leq N$ 的情况，只需要离散化一下所有的变化点到 f' 上，每次转移完之后 f' 的一整段的值一定是相同的，所以和之前一样维护即可。

2024 ICPC Hangzhou B. Barkley III

题目描述

维护一个数组 $a_{1 \sim n}$, q 次操作, 有如下几种操作:

- 1 l r x: $\forall l \leq i \leq r, a_i \leftarrow a_i \wedge x;$
- 2 s x: $a_s \leftarrow x;$
- 3 l r: 求:

$$\max_{k=l}^r \left\{ \bigwedge_{l \leq i \leq r, i \neq k} a_i \right\}$$

$1 \leq n, q \leq 10^6$, 值域 $[0, 2^{63} - 1]$ 。

解法思路

- 考虑每个二进制位，如果第 k 位在区间中有至少两个 0，那么这一位肯定没救了。
- 如果这一位在区间中全是 1，那么这一位在答案中也一定是 1。
- 如果这一位在区间中有恰好一个 0，那么如果 k 选中了这个 0，那么这一位就可以变成 1。
- 因此，我们贪心地会去选取最高的一位有恰好一个 0 的那一位。
- 考虑用线段树维护每一个二进制位中 0 的个数，反过来只需要使用线段树二分就能找到某一位那个唯一的 0，但是时间复杂度是 $O(n \log^2 V)$ 的，无法通过。
- 需要使用压位的方法，用两个 long long 分别表示区间中全为 1 的位置和区间中恰好有一个 0 的位置。通过与或取反的位运算操作计算。时间复杂度 $O(n \log V)$ 。

2025 ICPC EC Online (II) L. Xor Mirror

题目描述

维护序列 $A_{0 \sim N-1}$, 进行 Q 次操作, 有如下几种操作:

- 1 l r k: 对于 $i \in [l, r)$, 令 $B_i = A_{i \oplus k}$, 然后再对于 $i \in [l, r)$, 令 $A_i \leftarrow B_i$;
- 2 l r, 查询 $\sum_{i=l}^{r-1} A_i$.

$1 \leq N \leq 2^{18}$, 保证 N 是 2 的幂, $1 \leq Q \leq 2 \times 10^5$, $1 \leq A_i \leq 2^{20}$.
空间限制 64MB.

解法思路

- 先不考虑空间的限制，我们可以使用主席树维护序列 A 。
- 一次询问可以拆分为 $O(\log n)$ 个在主席树上对应一个区间的小区间。
- 但是注意需要打好下标异或的标记（主席树推荐使用标记永久化，否则空间消耗会很大）。
- 时空复杂度是 $O(q \log n)$ 的（实现得不好就会多一个 $\log$ ），接受不了。

解法思路

- 64MB 的空间限制怎么做？考虑定期重构，每 B 次询问重构一次（把 A 序列求出来重新建主席树）。
- 一次重构的时间是 $O(n)$ 的，最多只会进行 $O(\frac{n}{B})$ 次重构，重构的时间复杂度是 $O(\frac{n^2}{B})$ 的。
- 一次操作会增加 $O(\log n)$ 个主席树的节点，那么空间复杂度就是 $O(n + B \log n)$ 的。
- 取 $B = \frac{n}{\log n}$ 即可做到线性空间， $O(n \log n)$ 时间。
- 官方题解的做法是分块，时间复杂度是 $O(n\sqrt{n})$ 的。

LOJ 4258. 「KTSC 2024 R1」警察与小偷

题目描述

有一个 n 个节点的带边权的树，边权表示两个节点之间的距离。现在有 q 次询问，每次询问给出 x, v_1, y, v_2 ，表示警察从节点 x 出发，最大速度为 v_1 ，小偷从 y 出发，最大速度为 v_2 ，问警察在最坏情况下需要花多长时间抓到小偷。一些说明：

- 警察时刻都能知道小偷的位置，小偷不知道警察的位置；
- 警察和小偷在同一时刻处于相同位置（这个位置可以是某个节点，也可以是某条边的某个位置）时，认为警察抓住了小偷。

$2 \leq n \leq 10^5$ ， $1 \leq q \leq 10^5$ ，边权和最大速度均为 $[1, 10^6]$ 中的正整数。

CF1290E Cartesian Tree

题目描述

给定一个 $1 \sim n$ 的排列 $a_{1 \sim n}$ ，对于 $k = 1, \dots, n$ ，你需要求出，只保留 a 中不超过 k 的元素，剩下的元素相对位置关系不变，建出来一棵大根的笛卡尔树，求出这棵树所有节点子树大小的和。

$$1 \leq n \leq 1.5 \times 10^5。$$

解法思路

- 考虑依次插入每个元素，那么 i 的子树大小就是 $r_i - l_i - 1$ ，其中 l_i, r_i 是 i 向左/右第一个比 a_i 大的元素。
- 分别考虑求出 $\sum l_i$ 和 $\sum r_i$ ，发现用线段树维护的话，只需要支持，区间加，区间取 $\min$ 或 $\max$ ，那么使用吉司机线段树 (Segment Tree Beats) 即可解决。

洛谷 P8339 [AHOI2022] 钥匙

题目描述

给定一棵 n 个节点的树，每个节点都有一个宝箱或一把钥匙，宝箱和钥匙都有对应的颜色 c_i ，钥匙只能打开相同颜色的宝箱并且不能重复使用。同一种颜色的钥匙至多只有 5 把。

m 次询问，每次询问给出 s, t ，表示从 s 出发走到 t ，途中经过每个节点，如果是钥匙就带走，如果是宝箱，就尝试用已有的钥匙打开宝箱，求出能够打开多少个宝箱。

$1 \leq n \leq 5 \times 10^5$, $1 \leq m \leq 10^6$ 。

解法思路

- 考虑对于每种颜色独立计算贡献，可以把对应颜色的宝箱和钥匙建一棵虚树，但是仍然难以处理贡献。
- 考虑设置一个策略把钥匙和宝箱配对，就是每次使用最近获得的一把钥匙打开宝箱。
- 那么只需要对于每一把钥匙，从这个钥匙开始 dfs 一遍，维护一个栈就能够处理出这把钥匙对应了哪些宝箱。
- 再对于这样的一对钥匙宝箱，对应到 dfs 序平面的矩形加，再做一遍扫描线即可解决。

2022 ICPC Shenyang G. Meet in the Middle

题目描述

给定两棵 n 个节点的带边权的树 T_1, T_2 , 现在有 q 次询问, 每次询问给出 x, y , 求:

$$\max_{i=1}^n \{ \text{dist}_1(x, i) + \text{dist}_2(y, i) \}$$

$1 \leq n \leq 10^5$, $1 \leq q \leq 5 \times 10^5$, 边权在 $[1, 10^9]$ 范围内。

解法思路

- 考虑在 T_1 上点分治，每次处理 x, i 跨过分治中心的问题，维护每个子树在 T_2 上的直径。
- 每次询问查询除了这个子树之外的其他子树的直径的并，然后计算贡献即可。

P4220 [WC2018] 通道

题目描述

给定三棵 n 个节点的带边权的树 T_1, T_2, T_3 , 求:

$$\max_{u,v} \{ \text{dist}_1(u,v) + \text{dist}_2(u,v) + \text{dist}_3(u,v) \}$$

其中 $\text{dist}_k(u,v)$ 表示 u,v 在 T_k 上的最短路径长度。

$2 \leq n \leq 10^5$, 边权在 $[0, 10^{12}]$ 范围内。

解法思路

- 如果只有一棵树怎么做？求直径即可。
- 如果有两棵树怎么做？点分治 + 虚树求直径？点分治 + 直径合并？
- 三棵树怎么做？点分治 + 虚树 + 线段树合并维护直径。

HDU 7530 树上询问

题目描述

给定一棵 n 个节点的树，第 i 个节点点权为 p_i ，且 $p_{1 \sim n}$ 构成了 $1 \sim n$ 的一个排列。 m 次操作，有如下两种操作：

- 1 x y: 交换 p_x, p_y ;
- 2 l r: 询问是否存在树上的两个点 x, y 使得 x 到 y 路径上的点权恰好为 $[l, r]$ 区间内的所有整数。

$1 \leq n, m \leq 10^5$ 。

解法思路

- 本题解法众多，这里讲一种常数较小的方法。
- 考虑权值在 $[l, r]$ 内的点的 dfs 序最大的一个，如果是一条链，那么这个点一定是链的一端。
- 如果这条链的两端不是祖先关系的话，反着 dfs 一遍，就能求出链的另一端，这样链的两端就能够求出来了。
- 剩下的就是判断是否合法，只需要判断链上最大/最小值即可，使用树剖 + 线段树/全局平衡二叉树均可。
- 时间复杂度： $O(n + q \log^2 n) / O(n + q \log n)$ 。